

CSE235 Week 5: Functions

This week, we are discussing functions. We'll discuss using predefined functions provided by Python and how to define your own functions. As we move into creating more complex programs, we'll discuss how to break down a programming task into more manageable subtasks. We'll also introduce the concept of refactoring to improve already working code.

Predefined Functions

We've already been using some predefined functions in this class, such as `print()`, `input()` and type casting. Python offers many predefined functions both by default and in separate modules. Later, we'll also discuss installing external packages that provide additional functions and classes that you can use. To call a function, you write the name of the function followed by parentheses. The function may expect you to provide arguments, which are placed inside the parenthesis. For example, `print("Hello world")` is passing the string literal "Hello World" to the `print()` function as an argument. Some functions, like `print`, have optional arguments. Calling `print()` with no arguments simply prints a blank line to the screen.

Import

The `import` statement in Python is used to load external modules into your code. A module is simply another Python file. Importing the module allows the code written in the file to be used in the file you are currently writing. For example, Python provides the `math` module, which contains functions for common mathematical operations. Below is an example of importing the `math` module and using the `sqrt()` function to compute the square root of a number:

```
# importing math
import math

# when importing an entire module,
```

```
# you put the module name before the function,  
# with a period between them  
print(math.sqrt(4)) # prints 2
```

As seen in the example, the function being called and the module name are separated by a period. The module name is needed to tell the interpreter that you want to use the `sqrt()` function that is defined in the `math` module.

While not a problem with `math`, some module names can be long. To avoid needing to write long module names, Python provides a way to alias modules when you import them so that you don't have to type the full name.

```
# the "as m" section says to use the name "m" for math  
import math as m  
  
# now the alias m can be used instead of math  
print(m.sqrt(4)) # still prints 2
```

If you only need to use one or two functions from a module, then a `from` statement can be used to import only the functions you want. This also eliminates the need to use the module name to scope the functions:

```
# importing only sqrt  
from math import sqrt  
  
# now we don't need to specify that sqrt is from math  
print(sqrt(4)) # still prints 2
```

The limitation of using `from ... import ...` is that any additional functions you need from the module need to be added to the import list. If you `import ...`, then all the functions are immediately available, you just have to specify the module name each time you call them.

Programmer-Defined Functions

To define a function in Python, the following syntax is used:

```
def name(param1, param2):  
    statement
```

Functions begin with the `def` keyword, which is followed by the name and parameter list. The parameter list is a set of parentheses with 0 or more parameters. Parameters are names used for the values or variables that are passed to the function when it is called. The parameter list is followed by a colon. Just like the other statements (if, for, while, etc.), the body of the function is indented. The body of the function can contain any valid Python code (even other functions). Below is a basic example of an add function:

```
def add(a, b):  
    return a + b
```

This function has a return statement, `return a + b`. This means that the function `add()` will emit a value when it is called. Functions that do not return a value implicitly return the value `None`. The `None` value is Python's version of null or void. It is a value that indicates that there is no value (yes, it is a bit of an oxymoron). For example, in the following code snippet the function `print_greeting()` does not have a return statement. However, we can still use `print_greeting()` in an assignment statement. The variable will be assigned the value `None`, since `print_greeting()` does not return a value:

```
# this function does not have a return statement  
def print_greeting():  
    print("Hello World")  
  
# this is legal, the value None is stored in x  
x = print_greeting()  
print(x) # prints: None
```

Overloading (or lack thereof)

In many modern programming languages, function overloading is provided. Function overloading allows multiple functions with the same name to be defined. The functions are differentiated by having different parameter lists. For example, the following (with possible, slight syntactic variations) is valid in C++/Java/C#:

```
// this function has two parameters  
double computeTotal(double subtotal, double taxRate) {
```

```

    double taxAmount = subtotal * taxRate;
    return subtotal + taxAmount;
}

// this function has three parameters
// even though it has the same name as the previous function,
// the compiler can tell it's different since it has different parameters
double computeTotal(double itemPrice, int qty, double taxRate) {
    double subtotal = itemPrice * qty;
    return computeTotal(subtotal, taxRate);
}

// this function also has two parameters
// however, the second parameter in this function is int instead of double
// since the data type is different, the compiler can also tell the
// difference between this function and the first one
double computeTotal(double itemPriceWithTax, int qty) {
    return itemPriceWithTax * qty;
}

```

Unlike C++, Java, C#, and Visual Basic, Python is not compiled. As stated in week 1, Python is an interpreted language. Most interpreted languages (e.g., Python, PHP, JavaScript) do not support function overloading. Functions in these languages are tracked by their name only; the parameters are irrelevant. To get around this, you can either declare multiple functions with different names, or use default values and named arguments. Here is how you could achieve the same result as the above overloaded functions with default parameters:

```

# in Python, function overloading is not possible
# instead, you can provide default values for parameters
# if the parameter is optional, we can provide them a
# default value of None, and handle different cases
# within the function definition
def compute_total(price, tax_rate=None, qty=None):
    if tax_rate != None:
        tax_amount = price * tax_rate
        if qty != None:
            return tax_amount * qty
        else:
            return price + tax_amount
    elif qty != None:
        return price * qty
    else:
        return price

# price does not have a default value, so we should always
# provide a price as the first argument

```

```
# if we call the arguments in order, we don't need to specify
# the name of the parameter we are assigning a value
total_from_price_and_tax = compute_total(100, 0.07)
# if we want to skip past the tax rate, we can specify that
# qty=something as the second argument
total_from_price_and_qty = compute_total(10, qty=10)
# named arguments can be used to call the function with the
# parameters out of order. however, once you start assigning
# the parameters out of order, you have to specify the name
# for ALL the arguments
total_from_price_tax_and_qty = compute_total(tax_rate=0.07, qty=10, price=100)
# this would be illegal:
# out_of_order_call = compute_total(qty=10, 10, 0.07)
# SyntaxError: positional argument follows keyword argument
```

Alternatively, we can define three versions of the function with different names:

```
def compute_total(price, tax_rate):
    tax_amount = price * tax_rate
    return price + tax_amount

def compute_total_with_qty(price, tax_rate, qty):
    subtotal = price * qty
    return compute_total(subtotal, tax_rate)

def compute_total_with_qty_no_tax(price, qty):
    return price * qty

total_from_price_and_tax = compute_total(100, 0.07)
total_from_price_tax_and_qty = compute_total_with_qty(100, 0.07, 10)
total_from_price_and_qty = compute_total_with_qty_no_tax(100, 10)
```

Task Decomposition and Refactoring

Writing a program requires decomposing the task you have been given into more manageable sub-tasks. Then, you can solve the individual sub-tasks until you've created a program that solves the whole task you've been given. However, the resulting code of the program will often be messy, inefficient, and disorganized. Refactoring is the process in which you go through after writing the initial code and clean up the program. Next week, when we discuss testing and test driven development, you'll see a couple of longer examples of this. Here, I'll stick with a shorter

example.

Let's say that we have been given the task of writing a program that, given a range of integers, finds the integer that takes the most steps to reach 1 following the Collatz conjecture. The Collatz conjecture states the following: Given a positive integer, if it is even, divide it by 2. If it is odd, multiply it by 3 and subtract 1. Every integer will eventually reach 1 by doing this. This has never been formally proved, which is why it is called a conjecture and not a theorem, but there has never been an example to disprove it, either.

Let's break down the requirements of the program into manageable tasks:

1. Read two numbers
2. Apply the Collatz conjecture to each number
3. Count the number of steps to reduce the number to 1
4. Keep track of the number that took the most steps
5. Report the number that took the most steps to reach 1

With these tasks in mind, let's start implementing a solution. The easiest thing to do is to read the two numbers in, so let's do that:

```
print("This program will find which positive integer"  
      " in a given range takes the most steps to reach 1"  
      " following the Collatz Conjecture")  
  
start = int(input("Enter start of range: "))  
end = int(input("Enter end of range: "))
```

That's task 1 done. However, in writing this, I thought of a problem. While I am asking for the minimum and maximum number explicitly, the user could still enter them in the wrong order. They could also enter the same number twice. In that case, we'd only be checking one number. This is technically valid, so we'll allow the user to enter the same number, so the program can be used to check how many steps it takes to reduce a single number to 1. However, we should validate that the min is less than or equal to the max. We could allow the user to continually reenter numbers, but I'd rather just report the invalid usage and end the program. It is short program that only asks the user to enter two numbers, so forcing them to run it again to use it correctly is easier than writing the code to prompt the user to enter the correct

value.

```
print("This program will find which positive integer"
      " in a given range takes the most steps to reach 1"
      " following the Collatz Conjecture")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

# add logic to check that input is a valid range
if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)
```

Now, we can test if the program works. Below are some example inputs (the explanation message is excluded):

```
python3 collatz.py
Enter start of range: 1
Enter end of range: 5
python3 collatz.py
Enter start of range: 5
Enter end of range: 1
Invalid usage. Start must be less than end
python3 collatz.py
Enter start of range: 0
Enter end of range: 5
```

In testing it, I discovered another problem. 0 is not a valid input for the program. The input also cannot be 0 or negative. The conjecture does not apply to those values. Therefore, we need to eliminate the possibility that either element of the range is 0 or less.

```
print("This program will find which positive integer"
      " in a given range takes the most steps to reach 1"
      " following the Collatz Conjecture")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)

# add logic to check that 0 is not part of the range
if start <= 0 or end <= 0:
```

```
print("Invalid usage. Range must be positive")
exit(2)
```

Now, we can test it again:

```
python3 collatz.py
Enter start of range: 0
Enter end of range: 5
Invalid usage. Range must be positive
python3 collatz.py
Enter start of range: -4
Enter end of range: -1
Invalid usage. Range must be positive
```

Now, we've handled getting input from the user. Next, we need to apply the Collatz conjecture to each of them. First, let's loop through the numbers:

```
print("This program will find which positive integer"
      " in a given range takes the most steps to reach 1"
      " following the Collatz Conjecture")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)

if start <= 0 or end <= 0:
    print("Invalid usage. Range must be positive")
    exit(2)

for i in range(start, end + 1):
    print(i)
```

Before implementing anything else, we can check to make sure the loop works as intended:

```
python3 collatz.py
Enter start of range: 1
Enter end of range: 5
1
2
3
4
```



```
5
python3 collatz.py
Enter start of range: 500
Enter end of range: 500
500
```

Once we are satisfied, we can eliminate the print statement from the loop. Now, we need to write the algorithm for the Collatz conjecture. Rather testing it on each number in the range, we will comment out the for loop for now, and just run the algorithm on the first element of the range. That way, we can test if it works before running it on each number in the range.

```
print("This program will find which positive integer"
      " in a given range takes the most steps to reach 1"
      " following the Collatz Conjecture")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)

if start <= 0 or end <= 0:
    print("Invalid usage. Range must be positive")
    exit(2)

# for i in range(start, end + 1):
#     print(i)

value = start
while value != 1:
    if value % 2 == 0:
        value //= 2
    else:
        value = value * 3 + 1

print("value now equals", value)
```

We expect that for any number we enter, that start eventually becomes 1. Now, our third requirement states that we need to count the number of steps it takes for the number to reach 1. Again, before looping through each number in the range, let's implement the counting mechanism:

```

print("This program will find which positive integer"
      " in a given range takes the most steps to reach 1"
      " following the Collatz Conjecture")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)

if start <= 0 or end <= 0:
    print("Invalid usage. Range must be positive")
    exit(2)

# for i in range(start, end + 1):
#     print(i)

steps = 0
value = start

while value != 1:
    if value % 2 == 0:
        value //= 2
    else:
        value = value * 3 + 1
    steps += 1

print("value now equals", value)
print("it took", steps, "steps")

```

To verify that it is correct, we can hand calculate a couple of examples:

1

1 is 1. It takes 0 steps

5

1. 5 is odd. $5 * 3 + 1 = 16$
2. 16 is even. $16 / 2 = 8$
3. 8 is even. $8 / 2 = 4$
4. 4 is even. $4 / 2 = 2$
5. 2 is even. $2 / 2 = 1$
6. 1 is 1. It took 5 steps to get here

10

1. 10 is even. $10 / 2 = 5$

2. 5 is odd. But, we know from our previous work that it takes 5 steps to get 5 to 1. Therefore, it takes $1 + 5 = 6$ steps for 10 to get to 1.

13

1. 13 is odd. $13 * 3 + 1 = 40$
2. 40 is even. $40 / 2 = 20$
3. 20 is even. $20 / 2 = 10$
4. 10 is even. But, we know from previous work that it takes 6 steps to get 10 to 1. Therefore, it takes $3 + 6 = 9$ steps for 13 to get to 1.

Once satisfied with the number of test cases, we can run the program with them. Once satisfied that it works, we can move on to the next step, running the Collatz algorithm on all numbers in the range. To do so, we will move the code into the for loop, and replace `start` with `i`.

```
print("This program will find which positive integer"
      " in a given range takes the most steps to reach 1"
      " following the Collatz Conjecture")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)

if start <= 0 or end <= 0:
    print("Invalid usage. Range must be positive")
    exit(2)

for i in range(start, end + 1):
    steps = 0
    value = i
    while value != 1:
        if value % 2 == 0:
            value //= 2
        else:
            value = value * 3 + 1
        steps += 1

    print("value now equals", value)
    print("it took", steps, "steps")
```

Here is some example output for this point in development:

```
python3 collatz.py
Enter start of range: 5
Enter end of range: 6
value now equals 1
it took 5 steps
value now equals 1
it took 8 steps
```

Now, the requirements did not say to print out the steps for each number in the range. We are only supposed to print out the number of steps and the number which took the most steps. Therefore, we'll eliminate the print statements in the for loop that we used for testing, now that we are confident in the logic of the program. To store the number that took the most steps, and the number of steps, we'll need two more variables. Since these need to be used outside the while loop to report the final results at the end of the program, we need to declare them before the while loop.

The minimum possible number of steps is 0, which happens if the value is 1. Therefore, we assume that the maximum number of steps taken is 0 to start off with. Then, each time we run the Collatz algorithm on a number, we check if the number of steps taken was more than the current maximum. If it is, we update the maximum number of steps and the value that caused the maximum number of steps. This will allow us to save the max steps.

```
print("This program will find which positive integer"
      " in a given range takes the most steps to reach 1"
      " following the Collatz Conjecture")

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)

if start <= 0 or end <= 0:
    print("Invalid usage. Range must be positive")
    exit(2)

max_steps = 0
value_with_max_steps = start

for i in range(start, end + 1):
```

```

steps = 0
value = i
while value != 1:
    if value % 2 == 0:
        value //= 2
    else:
        value = value * 3 + 1
    steps += 1

if steps > max_steps:
    max_steps = steps
    value_with_max_steps = i

print("Most steps:", max_steps)
print("For value:", value_with_max_steps)

```

Once again, we will test the program to ensure it works. At this point, all requirements have been met. However, we can clean up the program a bit. First, let's move the greeting message to a separate function. It takes up several lines of the main function, without contributing anything meaningful to anyone reading the code:

```

def print_greeting():
    print("This program will find which positive integer"
          " in a given range takes the most steps to reach 1"
          " following the Collatz Conjecture")

print_greeting()

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

if start > end:
    print("Invalid usage. Start must be less than end")
    exit(1)

if start <= 0 or end <= 0:
    print("Invalid usage. Range must be positive")
    exit(2)

max_steps = 0
value_with_max_steps = start

for i in range(start, end + 1):
    steps = 0
    value = i
    while value != 1:
        if value % 2 == 0:

```

```

        value //= 2
    else:
        value = value * 3 + 1
        steps += 1

    if steps > max_steps:
        max_steps = steps
        value_with_max_steps = i

print("Most steps:", max_steps)
print("For value:", value_with_max_steps)

```

Next, let's move the error handling to a separate function. Recall that we can use the `exit()` function from `cstdlib` to exit the program if an error occurs. This will allow us to terminate the program from a non-main function:

```

def print_greeting():
    print("This program will find which positive integer"
          " in a given range takes the most steps to reach 1"
          " following the Collatz Conjecture")

def validate_input(start, end):
    if start <= 0 or end <= 0:
        print("Invalid usage. Range must be positive")
        exit(1)

    if start > end:
        print("Invalid usage. Start must be less than end")
        exit(2)

print_greeting()

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

validate_input(start, end)

max_steps = 0
value_with_max_steps = start

for i in range(start, end + 1):
    steps = 0
    value = i
    while value != 1:
        if value % 2 == 0:
            value //= 2

```

```

        else:
            value = value * 3 + 1
            steps += 1

    if steps > max_steps:
        max_steps = steps
        value_with_max_steps = i

print("Most steps:", max_steps)
print("For value:", value_with_max_steps)

```

Now, we can move the logic of performing the Collatz algorithm to a separate function. The only thing that we are interested in from the Collatz algorithm is how many steps it takes to complete, so we will have it return the number of steps:

```

def print_greeting():
    print("This program will find which positive integer"
          " in a given range takes the most steps to reach 1"
          " following the Collatz Conjecture")

def validate_input(start, end):
    if start <= 0 or end <= 0:
        print("Invalid usage. Range must be positive")
        exit(1)

    if start > end:
        print("Invalid usage. Start must be less than end")
        exit(2)

def count_collatz_steps(value):
    steps = 0
    while value != 1:
        if value % 2 == 0:
            value //= 2
        else:
            value = value * 3 + 1
        steps += 1
    return steps

print_greeting()

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

validate_input(start, end)

```

```

max_steps = 0
value_with_max_steps = start

for i in range(start, end + 1):
    steps = count_collatz_steps(i)

print("Most steps:", max_steps)
print("For value:", value_with_max_steps)

```

We can also move printing the results to a separate function:

```

def print_greeting():
    print("This program will find which positive integer"
          " in a given range takes the most steps to reach 1"
          " following the Collatz Conjecture")

def validate_input(start, end):
    if start <= 0 or end <= 0:
        print("Invalid usage. Range must be positive")
        exit(1)

    if start > end:
        print("Invalid usage. Start must be less than end")
        exit(2)

def count_collatz_steps(value):
    steps = 0
    while value != 1:
        if value % 2 == 0:
            value //= 2
        else:
            value = value * 3 + 1
        steps += 1
    return steps

def print_results(steps, value):
    print("Most steps:", steps)
    print("For value:", value)

print_greeting()

start = int(input("Enter start of range: "))
end = int(input("Enter end of range: "))

validate_input(start, end)

```



```
max_steps = 0
value_with_max_steps = start

for i in range(start, end + 1):
    steps = count_collatz_steps(i)

print(max_steps, value_with_max_steps)
```

We've now made `main()` function cleaner, with logic that is easier to follow.

Summary

This week, we discussed functions. First, we discussed importing modules and using predefined functions. Next, we discussed how to define and use our own functions in Python. Finally, methods for decomposing a programming problem into manageable tasks was presented. We provided an example task decomposition for a problem, then refactored the code into a separate functions to clean up the mainline logic of the program.